



Individual Assignment

Shameems Cosmetics

Level: 06

CB010090 - Shumair Ahamadh

COMP6001

Decision Analytics 2

Batch Code/s: HF2521COM

Hand Out Date: 28 July 2025

Hand In Date: 05 September 2025

Weightage: 50%

INSTRUCTION TO CANDIDATES:

1. Students are advised to underpin their answers with the use of references (cited using the Harvard Name System of Referencing).
2. Late submission will be awarded zero (0) unless Exceptional Circumstances (EC) are upheld.
3. We draw your attention to the [Academic Conduct](#) Procedure to encourage good academic practice.
4. Assignment presentation and modeling work should be submitted as a softcopy

Acknowledgment

I would like express my heartfelt gratitude to my lecturer in Decision Analytics, Dr. Rajitha Navarathne, who has guided, given constructive feedback, and encouraged me throughout the course of this module. His knowledge and skills have contributed a lot towards improving my work in Decision Analytics module.

I would also like to express my heartfelt gratitude to Mr. Mohammed Anees, the owner of Shameems Cosmetics, who shared such insightful business information and useful knowledge that contributed to the formation of the practical implementation of this project in the real world.

I am also very grateful to APIIT Sri Lanka because it offered me the academic materials, learning facilities, and technical assistance that enabled the completion of this study to be successful. Finally, I would like to thank my friends and family who were able to motivate me, be patient, and support me at all times, which were invaluable in this assignment.

Abstract

This report involves the design and development of a centralized management system for Shameems Cosmetics, a retail business that deals with cosmetics, personal care, and household products. The organization was used to using manual record keeping, which resulted in inefficiency in managing customers, monitoring stocks, and making business decisions. MERN stack solution was adopted to overcome these challenges and is supported by a well-organized MongoDB database. The system has built customer, employee, product, order, promotion, and administrator collections that ensure seamless business operations with validation, indexing, and analytics aggregation pipelines. In the application, transparency and efficiency are guaranteed by the presence of a secure log-in, an interactive dashboard, and CRUD functionalities in various modules. Business insights such as top-selling products, loyalty customers, and active promotions are derived in real time to guide managerial decisions. The integration of modern web technologies with data analytics will be used to show how small businesses can increase the effectiveness of their operations and implement evidence-based strategies to grow.

Contents

Acknowledgment	i
1 Introduction	7
1.1 Background Study	7
1.2 Market Position	7
1.3 Problem Context	8
1.3.1 Record Keeping Inefficiencies	8
1.3.2 Stock Management and Expiry Losses	8
1.3.3 Customer Loyalty Tracking	8
1.4 Key Business Questions	9
1.5 Link to Data Engineering Pipeline	9
2 Data Engineering Pipelines	11
2.1 What is Data Pipeline?	11
2.1.1 Overview of Data Pipelines	11
2.1.2 Importance of Efficient Data Pipelines for Business	12
2.2 Database	12
2.2.1 Relational Databases	12
2.2.2 NoSQL Databases	12
2.2.3 Relational Database vs NoSQL Databases	13
2.2.4 Importance of MongoDB for this project	13
3 MongoDB	14
3.1 History of MongoDB	15
3.2 Benefits of MongoDB	15
3.3 Limitations of MongoDB	16
4 MongoDB Database Design and Development	17
4.1 Database Design	17
4.2 Data Model Diagram	18

4.2.1	How the Data Model Supports the Business Goals	20
4.3	Database Collection Design	20
4.3.1	Customers	20
4.3.2	Order	20
4.3.3	Employees	22
4.3.4	Products	22
4.3.5	Promotions	23
4.3.6	Admin	24
4.4	MongoDB creation using MongoDB Shell	24
4.5	MongoDB Compass	31
4.5.1	Database Development in MongoDB	32
5	MERN Stack Development	37
5.1	Overview	37
5.1.1	How the MERN Stack Works	37
5.1.2	Advantages of MERN Stack	37
5.2	Shameems Cosmetics Dashboard System	38
5.2.1	Interfaces	39
6	Data Analysis and Insights	46
6.1	Relevance to the Shameems Cosmetics	46
6.2	Solving the Business Problem	46
6.2.1	Customer Retention	47
6.2.2	Stock Management	47
6.2.3	Sales Trend	47
6.2.4	Employees Performance	47
6.2.5	Promotional Effectiveness	47
6.2.6	Financial Insights	47
6.3	Key Questions and Queries	48
6.3.1	Question 1: What are the top-selling products?	48
6.3.2	Question 2. Who are the most loyal customers based on the loyalty points?	49
6.3.3	Question 3. What is the total number of customers?	50
6.3.4	Question 4. How many employees are currently active, and what roles do they perform?	51
6.3.5	Question 5. Which product categories have the highest average price?	51
6.4	Hypothesis	53
6.4.1	Hypothesis 1: Promotions increase order frequency within a period	53

6.4.2	Hypothesis 2: Certain product categories generate the highest revenue?	54
6.4.3	Hypothesis 3. Top brands hold atleast 50% of the total stock units?	56
7	Conclusion	58

List of Figures

1.1	Shameems Cosmetics Logo	7
3.1	MongoDB Logo	14
4.1	MongoDB Data Model for Shameems Cosmetics	19
4.2	Sample Document Structure of the Customers Collection	20
4.3	Sample Document Structure of the Order Collection	21
4.4	Sample Document Structure of the Employees Collection	22
4.5	Sample Document Structure of the Products Collection	23
4.6	Sample Document Structure of the Promotions Collection	23
4.7	Sample Document Structure of the Admin Collection	24
4.8	Checking available databases	25
4.9	Switch databases	25
4.10	Active databases	26
4.11	Create database collections	26
4.12	Show database collections	27
4.13	Inserting a Single Document into a Collection 'Customers'	27
4.14	Inserting a Many Documents into a Collection 'Customers'	28
4.15	Find Documents in Collection 'Customers'	29
4.16	Update price in Collection 'Products'	30
4.17	Update the stock in Collection 'Products'	30
4.18	Delete the Promotion record of 'PROMO003' in 'Promotions' collection .	31
4.19	User Interface of MongoDB Compass	32
4.20	Creating a new connection in MongoDB Compass	32
4.21	Creating a new collection in MongoDB Compass	33
4.22	Creating a Database in MongoDB Compass	33
4.23	Importing Documents in the format of JSON or CSV files	34
4.24	View the inserted records	34
4.25	Updating phone number Field	35
4.26	Find documents with filter options	35
4.27	Delete a specific document	35

4.28	Drop a specific collection	36
5.1	Shameems Cosmetics Login Page	38
5.2	Shameems Cosmetics Dashboard	39
5.3	Shameems Cosmetics Customers Page	40
5.4	Shameems Cosmetics Order Page	41
5.5	Shameems Cosmetics Products Page	42
5.6	Shameems Cosmetics Employees Page	43
5.7	Shameems Cosmetics Promotions Page	44
5.8	Shameems Cosmetics Admin Page	45
6.1	Question 1. Top-selling Products (Query)	48
6.2	Answer 1. Top-selling Products (Query)	49
6.3	Question 2. Loyal Customers based on loyalty points	50
6.4	Question 3. Total number of customers.	50
6.5	Question 4. Active employees and roles.	51
6.6	Question 5. Products with high average price.	52
6.7	Hypothesis 1: Question	53
6.8	Hypothesis 1: Answer	54
6.9	Hypothesis 2: Question	55
6.10	Hypothesis 2: Answer	55
6.11	Hypothesis 3: Question	56
6.12	Hypothesis 3: Answer	56

List of Tables

2.1	Comparison of Relational vs. NoSQL Databases	13
-----	--	----

Chapter 1

Introduction

1.1 Background Study

Shameems Cosmetics is a well-established retail business located at No.38, Dalada Veediya, Kandy, Sri Lanka. Founded in 2008, it has become a reliable retailer in Sri Lanka's central region. that deals with cosmetics, personal care, and household goods. It has been able to build a loyal client base over the years as it offers a wide variety of good quality products and customer service. Its brand puts it at the forefront of the retail industry in Kandy.



Figure 1.1: Shameems Cosmetics Logo

1.2 Market Position

Shameems Cosmetics is a highly regarded Kandy cosmetic store. The store itself has been in operation on Dalada Veediya for many decades, which is an indicator of its toughness and the confidence that its customers have in them. Shameems has numerous brands of cosmetics, including a large number of imported brands to meet the different needs

of customers. It has developed a good competitive advantage by providing high-quality products, including skincare and household products. In addition, its established location on one of Kandy's busiest streets is an added advantage in attracting regular customers, walk-in customers, and foreign customers.

1.3 Problem Context

Several challenges of Shameems regarding the record keeping.

1.3.1 Record Keeping Inefficiencies

The store continues to maintain manual records of its operations, as a result of which, the records are often erroneous and inconsistent. The data of customers, employees and sales records is distributed among loose files and hand-written books. Due to this fragmented system, it becomes difficult to monitor the history of customers, identify trend of orders, and conduct loyalty programs effectively.

1.3.2 Stock Management and Expiry Losses

The absence of automated system to keep track of the product expiry dates and stock level is one of the biggest challenges. Failure to detect the items in time leads to wastage and loss of money as they are left to expire. Also, there is a problem of not being able to monitor low stock or high moving products thus making it harder to manage inventory.

1.3.3 Customer Loyalty Tracking

Shameems boasts of high numbers of loyal customers. However, this is hampered by the fact that there is no digital system of tracking loyalty to reward and retain them. The use of loyal points and discounts is usually done manually, and it creates mistakes and discrepancies. Such errors may damage consumer satisfaction and decrease the number of repeat purchases.

Effect on Business: All these problems are impairing the efficiency and growth of operations. Poor decision-making, lapsed stock increases the costs, and lack of organized loyalty program reduces customer retention are some of the effects of inaccurate records. The absence of an adequate data-driven system means that the business will lose its competitive advantage in a more digital retailing environment.

1.4 Key Business Questions

The business questions, which are deemed critical to the development of a data-driven solution, include the following:

Q1 Which Products are top-selling and contributes most to the revenue?

Answer: This helps prioritizing the stock levels, promotional campaigns and ensuring the availability of the fast moving items.

Q2 Which customers are more loyal and generate repeat sales?

Answer: Identifying loyal customers allows for targeted loyalty rewards and special offers to improve retention.

Q3 What are the seasonal or monthly trends?

Answer: Detecting purchase patterns helps in planning inventory and marketing strategies aligned with festive seasons and peak demand periods..

Q4 How much financial loss occurs due to expired or wasted stocks?

Answer: The measurement of losses would justify the use of expiry tracking and automated warning systems.

Q5 Which employees are driving the most sales or transactions?

Answer: Employee performance metrics can inform training needs and incentive programs.

Q6 How effective are promotional campaigns in boosting sales?

Answer: Linking promotions with sales data enables evaluating return on investment and adjusting future campaigns..

Q7 What is the average order value and how can it be increased?

Answer: Measuring AOV helps in designing cross-selling and upselling strategies to maximize revenue per transaction.

1.5 Link to Data Engineering Pipeline

The operations of modern retailing companies are based on the organized streams of data to be efficient, promote real-time decision-making, and guarantee reliability. For Shameems Cosmetics, a data engineering pipeline offers a methodically structured capturing, processing, and storing of data and analyzing business data. This makes sure that raw transactions and operating data are converted into practical knowledge.

The proposed pipeline includes the following stages:

- **Data Ingestion:** The web interface and point-of-sale system collect customer information, sales of products, employee operations, and promotions.
- **Transformation:** The data input is verified, cleansed, and organized. Some of these examples are, elimination of duplicates, the expiry date must be formatted correctly and standardization of products categories.
- **Storage:** The purged data is stored in MongoDB collections, which allow managing the schema and quickly retrieving the records.
- **Analysis:** Top products, loyal customers, and monthly sales patterns are some of the insights using MongoDB aggregation pipelines.
- **Visualization:** The owner is given the results on the dashboard to monitor the actual performance in real time and make decisions.

Shameems Cosmetics will be able to move out of reactive management by applying this pipeline to preemptive decision making. Likewise, loyalty tracking ensures that customers are rewarded and praised in a timely manner. This unification of data pipelines fills the analytical insights processes of operations, which puts the company in a position to embrace predictive models and innovative analytics in the future. Finally, it turns the store into a data-driven enterprise, improving competitiveness in the new retail environment.

Chapter 2

Data Engineering Pipelines

2.1 What is Data Pipeline?

A data pipeline is a structured process that enables the smooth flow of data from its source to its destination, where it can be stored, analyzed, and visualized. It ensures that raw information produced by business processes will be refined, processed, and presented in a consistent and trusted form to enable organizations to extract usable knowledge and aid in decision-making (Gonzalez, White, and Davis 2020; Provost and Fawcett 2013).

2.1.1 Overview of Data Pipelines

The data pipeline for Shameems Cosmetics follows well-established:

- **Data Ingestion:** Storing customer transactions, employee activities, promotions, and changing inventory using the application.
- **Data Transformation:** Cleaning, verifying and organizing raw data to eliminate any discrepancy like duplicates of a record or the wrong product line.
- **Data Storage:** Persisting an organization of data into MongoDB collections fast accessibility and scalability.
- **Data Analysis:** Executing queries and aggregation pipelines to create knowledge about the best customers, sales, and performance of products.
- **Data Visualization:** Providing analytics in the form of dashboards and reports to make decisions in business in real time.

All these steps will make the system more reliable and it will be possible to ensure that the data will be accurate, consistent, and useful in the operational and strategic scope.

2.1.2 Importance of Efficient Data Pipelines for Business

Retail companies that are moving towards data-driven are heavily dependent on efficient data engineering pipelines. Organizations like Spa Ceylon have proven the usefulness of organized pipes in e-Commerce as a strategy to track the behavior of customers and maximize stock (Laudon and Traver 2023). In the context of Shameems Cosmetics, pipelines serve the following purposes:

- Enable real-time monitoring of the stocks.
- Improve customer engagement by analyzing purchase history and loyalty data.
- Provide financial insights through order trends and average order values.
- Support scalability as the business expands its operations.

Thus, an efficient pipeline bridges operational workflows with analytics, making the organization agile and competitive.

2.2 Database

2.2.1 Relational Databases

Relational databases also called RDBMS store data in form of structured tables comprised of rows and columns. They are also consistent and also, they are strict and therefore suitable in applications that require transactional integrity with the help of ACID model. The most typical ones are MySQL, PostgreSQL and Oracle.

Strengths: Schema consistency, reliability and data integrity.

Weaknesses: Rigid schema design and poor scalability in handling high dynamic or unstructured data (Coronel and Morris 2019).

2.2.2 NoSQL Databases

NoSQL databases are designed for flexibility, scalability, and handling unstructured data.

Types include:

- **Document Stores:** e.g., MongoDB, store JSON-like documents
- **Key-value Stores:** e.g., Redis, optimized for fast lookups.
- **Column Stores:** e.g., Cassandra, effective for time series data.

- **Graph Databases:** e.g., Neo4j, ideal for relationship based data.

Strengths: Schema flexibility, high scalability, and performance with large data volumes.

Weaknesses: Weaker consistency (eventual consistency) and less mature tooling compared to RDBMS (Sadalage and Fowler 2012).

2.2.3 Relational Database vs NoSQL Databases

Feature	Relational DB	NoSQL DB
Schema Design	Fixed schema, predefined tables	Schema-less, flexible document or key-value models
Data Consistency	Strong consistency (ACID)	Eventual consistency (BASE)
Scalability	Vertical scaling (add CPU/RAM)	Horizontal scaling (sharding, replication)
Data Types	Structured, tabular data	Unstructured/semi-structured data (JSON, XML)
Best Use Case	Banking, HR, financial systems	E-commerce, IoT, social media platforms

Table 2.1: Comparison of Relational vs. NoSQL Databases

2.2.4 Importance of MongoDB for this project

The best fit to Shameems Cosmetics is MongoDB which is a document based NoSQL database. It has a flexible schema that is compatible with the MERN stack, and it has an in-built aggregation framework that provides real-time analytics. Horizontal scaling allows the system to increase with customer and product data. Also, the indexing, embedded documents, and \$lookup queries of MongoDB provide high performance as well as profound analytics (Chodorow 2013; MongoDB Inc. 2024).

Chapter 3

MongoDB

MongoDB is a popular NoSQL database that can be used to store, query and manipulate massive amounts of unstructured and semi-structured data. In contrast to the conventional relational databases, it is based on a document-oriented model, which provides flexibility and scalability to modern applications to the developers. It is more likely to be used with web-based systems like Shameems Cosmetics because of its seamless integration with the MERN stack (Chodorow 2013; Sadalage and Fowler 2012).



Figure 3.1: MongoDB Logo

3.1 History of MongoDB

MongoDB was first developed by a company named 10gen (now MongoDB Inc) in 2007 and the first public release was on 2009. It was created to address the weaknesses of relational databases with web-scale applications. MongoDB is one of the most popular NoSQL databases in the world nowadays.

Key Events

- **2007:** Development of MongoDB by 10gen by focusing of high scalability and schema flexibility.
- **2009:** First open-source release which gained more popularity.
- **2010:** Adopted by major companies such as eBay and Adobe for large-scale web applications.
- **2017:** MongoDB was listed in the NASDAQ and trades as MDB.
- **2020s:** Today, MongoDB is popular in e-commerce, medical, and IoT industries where its data requirements are high-performance.

3.2 Benefits of MongoDB

- **Schema Flexibility:** MongoDB is a document-oriented database where documents are represented in a JSON-like format and are not rigidly schematized. This allows the developers to add features to an application without redesigning the whole database schema (Chodorow 2013)
- **Horizontal Scalability:** MongoDB distributes data among a number of servers by sharding. This way enables the system to scale and be able to perform as the data set increases.
- **High Availability:** Replica sets enable the provision of several copies of data and automatic fail over. Consequently, the database remains alive even in case of failure of individual servers.
- **Rich Query Language:** The query language and the aggregation framework of MongoDB are effective analysis tools. They also allow users to execute real-time analytics and complicated transformations directly within the database.
- **Integration with MERN:** The M component in the MERN stack is MongoDB. It is compatible with Node.js and JavaScript, which facilitates the creation of web

applications in a short time-period (MongoDB Inc. 2024).

- **Performance:** MongoDB is optimized to handle large volumes of reads and writes, hence it is able to handle applications, which require real-time data processing effectively.
- **Community and Ecosystem:** The good community support, well-documented, and cloud solutions such as MongoDB Atlas enhance usability and assist businesses to adopt the platform.

3.3 Limitations of MongoDB

- **Eventual Consistency:** Unlike traditional relational databases, MongoDB does not focus on the immediate consistency but on the aspect of performance and scalability.
- **Memory Consumption:** Mostly dynamic schemas and vast indexing consume more memory as compared to relational databases.
- **Complex Transactions:** Despite the fact that MongoDB has multi-document transaction, it is not as efficient as the traditional relational system (Stonebraker 2010).
- **Data Duplication:** The de-normalization approach in document storage can sometimes lead to redundancy and larger database sizes.
- **Steeper Learning Curve:** The query language and the aggregation pipelines used by MongoDB may seem more challenging at first to SQL-experienced developers.

Chapter 4

MongoDB Database Design and Development

Any data-driven application is based on database design. For Shameems Cosmetics, MongoDB as a documents based NoSQL database provides flexibility, performance, and scalability in the processing of customer, employee, product, and order-related data. Unlike traditional relational databases, MongoDB does not exclude schema-less storage and it is incredibly helpful in the context of retail environment where the attributes of products and promotions may be altered, quite often.

A well-structured design was implemented to ensure efficient CRUD operations and support for analytics such as identifying top customers, monitoring sales performance, and tracking stock expiration. The design of this database is modular collections that minimize redundancy, the easy querying of these collections via aggregation pipelines and align with the long-term growth requirements of the business.

4.1 Database Design

Designing of Collections

The system was designed with six main collections, each addressing specific business needs:

- **Customers:** Stores customer details, including name, email, phone, gender, loyalty points, and account status. This collection is key for customer relationship management and loyalty programs.
- **Orders:** Portrays the information about customer purchases that contains links to products, employees, and promotions. Orders consist of items, quantities, total

amount, payment method, order date, and references to other related entities.

- **Employees:** Keeps records of employees, including full name, address, age, salary, position, within the firm date of employment and status. Provides access according to roles (cashier, sales, stock organizer, accountant).
- **Products:** It has a product description, which consists of the name, category, brand, selling price (LKR), stock quantity, and the expiry date. Allows the tracking of stocks and expiry.
- **Promotions:** Handles promotions that are in-store and those that have expired and details include; title, description, value of the discount, commencement date and expiration date and the product types that the promotions apply to.
- **Admin:** Includes system administrator and owner information with log in credentials (username, password hash). Offers system control authentication and access control.

4.2 Data Model Diagram

Overview of Data Model Diagram

The finalized data model, shown in Figure 4.1, illustrates the relationships between collections. It captures the essential operations of Shameems Cosmetics while ensuring data integrity and business scalability.

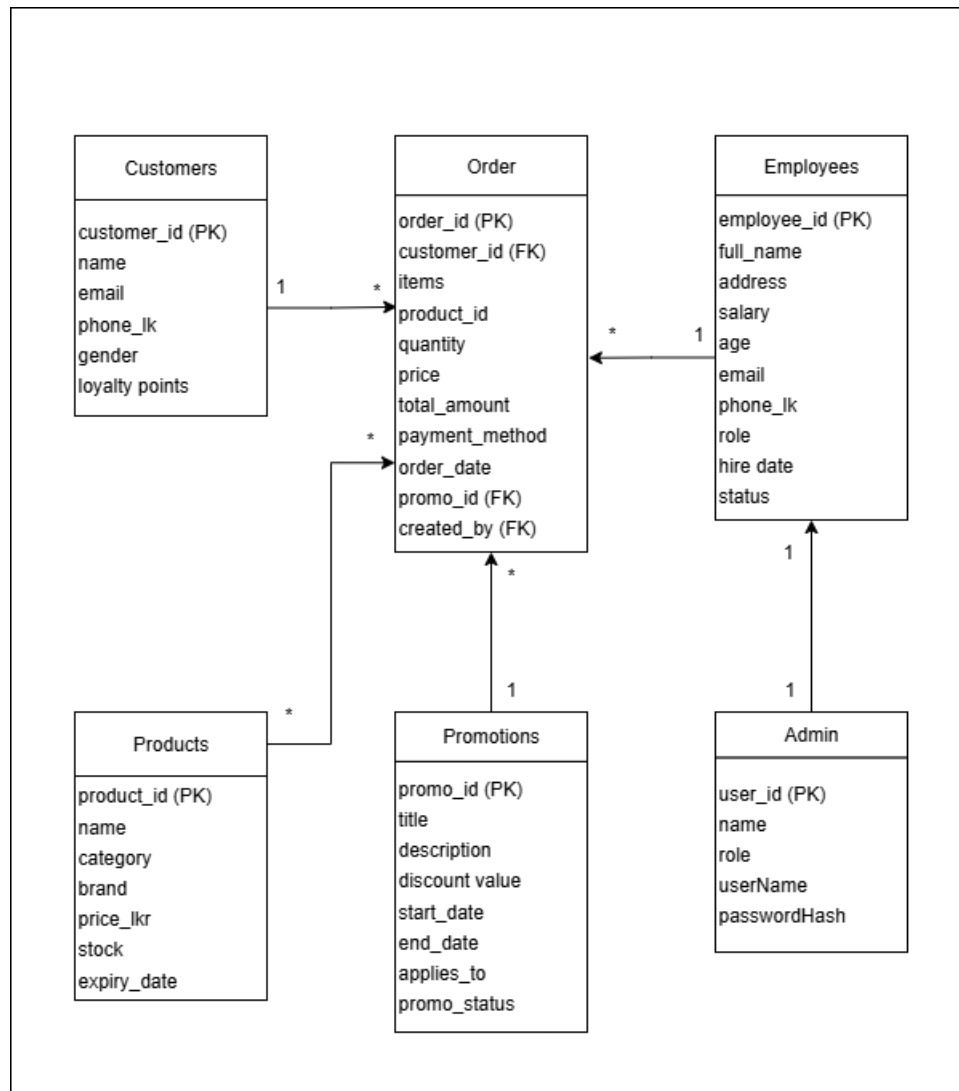


Figure 4.1: MongoDB Data Model for Shameems Cosmetics

Relationships in the Data Model

- **Customers:** can place multiple orders, creating a one-to-many relationship between Customers and Orders.
- **Order:** can include multiple products, and each product can appear in multiple orders, forming a many-to-many relationship.
- **Employees:** are linked to Orders as the staff responsible for processing transactions.
- **Products:** are linked to orders as the products which the customers purchased
- **Promotions:** are tied to Orders, allowing discounts to be applied and tracked at the order level.

- **Admin:** manage system access and oversee employee and order records, ensuring secure system governance.

4.2.1 How the Data Model Supports the Business Goals

The database model was structured as a storage model as well as to help the business make actionable insights and streamline workflows. The collections help the company to achieve the most important objectives of the retail business, including customer retention, transparency in operations, and effective promotions.

4.3 Database Collection Design

4.3.1 Customers

- **customer_id:** Unique string identifier of a customer
- **name:** Name of the customer
- **email:** E-mail of the customer
- **phone_lk:** Contact number of the customer
- **gender:** gender of the customer
- **loyalty_points:** Customer loyalty points based on how much they have bought products

```
_id: ObjectId('68b4558358bf7f897a0a25b4')
customer_id : "CUS0001"
name : "Upali Rathnayake"
email : "upali.rath@gmail.com"
phone_lk : 772336583
gender : "Male"
loyalty_points : 10
```

Figure 4.2: Sample Document Structure of the Customers Collection

4.3.2 Order

- **order_id:** Unique string identifier of a order
- **customer_id:** Unique string identifier of a customer which refers to customer collection

- **items:** An array of the products they have bought.
- **product_id:** Unique string identifier of a product which refers to product collection
- **quantity:** Quantity of the product
- **price:** Price of the product
- **total_amount:** Total Price of the items purchased
- **payment_method:** Payment method which was used to buy the products
- **order_date:** Date of the order initiated
- **promo_id:** Unique string identifier of a promotion which refers to promotion collection
- **created_by:** Unique string identifier of a employee which refers to employee collection

```
_id: ObjectId('68b4690e58bf7f897a0a2722')
order_id: "ORD001"
customer_id: "CUS0016"
▼ items: Array (3)
  ▼ 0: Object
    product_id: "PRD027"
    quantity: 2
    price: 1800
  ▼ 1: Object
    product_id: "PRD058"
    quantity: 3
    price: 1050
  ▼ 2: Object
    product_id: "PRD143"
    quantity: 3
    price: 7100
total_amount: 28050
payment_method: "Card"
order_date: "2025-08-18T18:56:47Z"
promo_id: null
created_by: "EMP002"
```

Figure 4.3: Sample Document Structure of the Order Collection

4.3.3 Employees

- **employee_id**: Unique string identifier of a employee.
- **full_name**: Full name of the employee.
- **address**: Address of the employee.
- **salary_lkr**: Salary of the employee.
- **age**: Age of the employee.
- **email**: Email of the employee.
- **phone_lk**: Contact number of the employee.
- **role**: Role of the Employee in the business (eg:- Cashier, Accountant, Sales and Stock Organizer).
- **hire_date**: Date of the employee hired.
- **status**: Employment status (active/inactive).

```
_id: ObjectId('68b45d9a58bf7f897a0a26e8')
employee_id: "EMP001"
full_name: "Supun Bandara"
address: "Katugastota Road, Kandy"
salary_lkr: 34000
age: 48
email: "supunbandara@outlook.com"
phone_lk: 759081935
role: "Cashier"
hire_date: 2015-02-27T00:00:00.000+00:00
status: "active"
```

Figure 4.4: Sample Document Structure of the Employees Collection

4.3.4 Products

- **product_id**: Unique string identifier of a product.
- **name**: Name of the product.
- **category**: Product Category (Men's Perfume, Shampoo, Conditioner)
- **brand**: Brand of the product.

- **price_lkr:** Price of the product
- **stock:** Quantity available.
- **expiry_date:** Expiration tracking to minimize waste.

```
_id: ObjectId('68b457a158bf7f897a0a2631')
product_id: "PRD003"
name: "Eros 50ml"
category: "Men's Perfumes"
brand: "Chandanalepa"
price_lkr: 21500
stock: 17
expiry_date: 2028-07-31T00:00:00.000+00:00
```

Figure 4.5: Sample Document Structure of the Products Collection

4.3.5 Promotions

- **promo_id:** Unique identifier.
- **title:** Unique identifier.
- **description:** Description of the promotion.
- **discount_value:** Discount Value
- **start_date:** Start date of promotion
- **end_date:** End date of promotion
- **applies_to:** Category or product reference.
- **promo_status:** Enum (upcoming, active, expired).

```
_id: ObjectId('68cc3a36b295bae0d8cc2596')
promo_id: "PROM0001"
title: "Christmas - Mega Sale"
description: "5% Discount on all products for Christmas month"
discount_value: "5"
start_date: 2025-12-01T00:00:00.000+00:00
end_date: 2025-12-31T00:00:00.000+00:00
applies_to: "All Products"
promo_status: "upcoming"
```

Figure 4.6: Sample Document Structure of the Promotions Collection

4.3.6 Admin

- **user_id:** Unique identifier.
- **name:** Name of the User
- **role:** Owner or admin privileges
- **username:** User name used for login
- **passwordHash** Encrypted password for secure login.

```
_id: ObjectId('68c43eb955904cddcb0cc0c4')
user_id : "USER001"
name : "Mohammed Anees"
role : "Owner"
username : "AneesOwner1"
passwordHash : "$2a$10$roz1l.nIh/AKKN9LrR7EmOWr4EXzkbQRfHoonskaVIDDAqF3oIZwq"
```

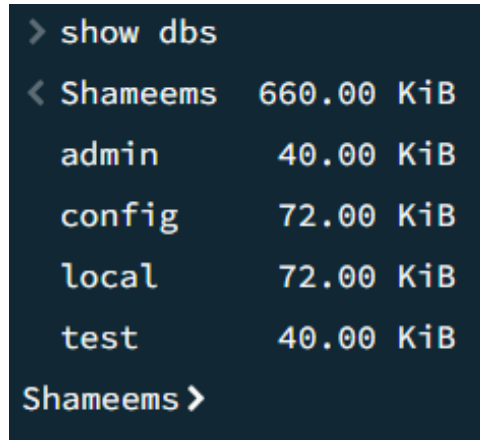
Figure 4.7: Sample Document Structure of the Admin Collection

4.4 MongoDB creation using MongoDB Shell

The MongoDB Shell (mongosh) offers a command-line utility that is used to create manage and query databases. Developers commonly use it in initial setup, testing, and data manipulation, and later integrate it with a front-end application. The steps that were followed to create and populate the Shameems Cosmetics database using MongoDB Shell include the following:

Checking Available Databases

`show dbs`



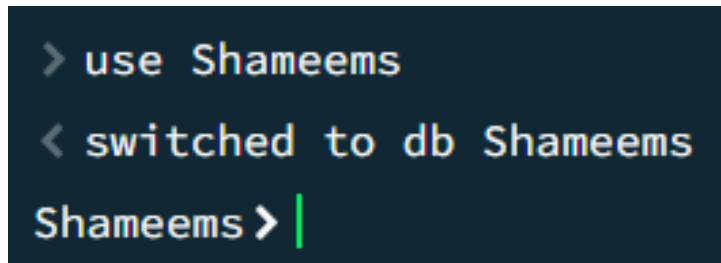
```
> show dbs
< Shameems  660.00 KiB
    admin    40.00 KiB
    config   72.00 KiB
    local    72.00 KiB
    test     40.00 KiB
Shameems >
```

Figure 4.8: Checking available databases

This command lists all databases currently available on the MongoDB server.

Switching to the Shameems Database

`use shameems`



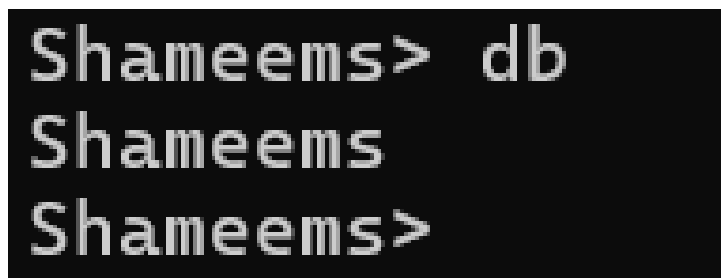
```
> use Shameems
< switched to db Shameems
Shameems > |
```

Figure 4.9: Switch databases

This command switches to the database named `Shameems`. If it does not exist, MongoDB will create it.

Verifying the Active Database

db

A terminal window with a black background and white text. The prompt 'Shameems>' is followed by 'db', then 'Shameems' on the next line, and 'Shameems>' on the third line.

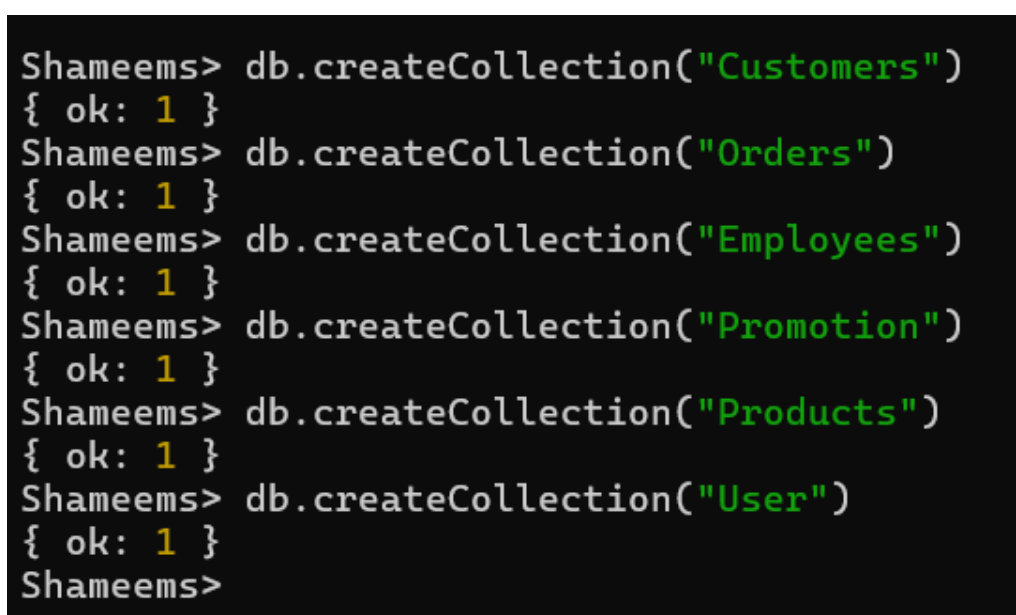
```
Shameems> db
Shameems
Shameems>
```

Figure 4.10: Active databases

Displays the name of the active database.

Creating Collections in the Database

```
db.createCollection("Customers")
db.createCollection("Employees")
db.createCollection("Products")
db.createCollection("Orders")
db.createCollection("Promotion")
db.createCollection("User")
```

A terminal window with a black background and white text. The prompt 'Shameems>' is followed by six lines of 'db.createCollection' commands for 'Customers', 'Orders', 'Employees', 'Promotion', 'Products', and 'User'. Each command is followed by a response '{ ok: 1 }'. The final line is 'Shameems>'.

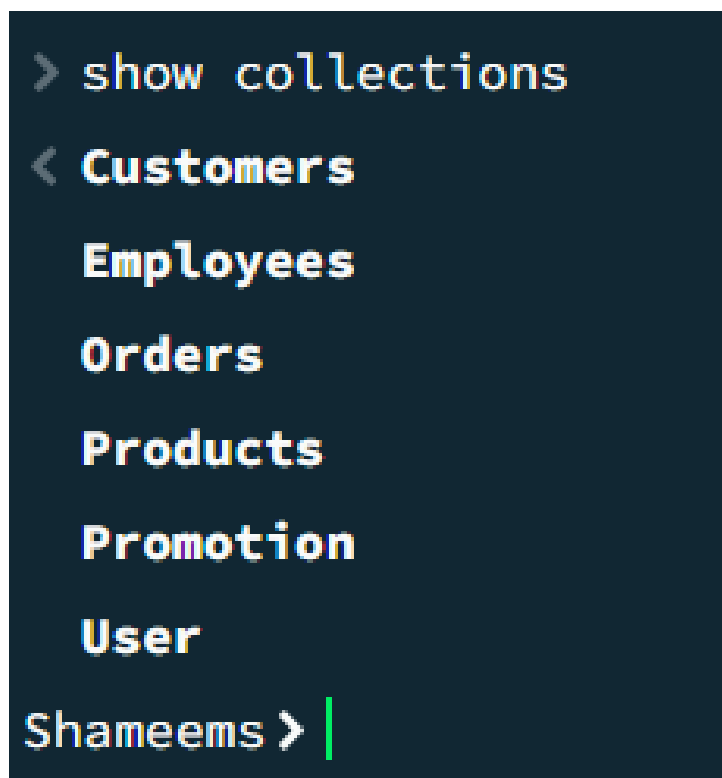
```
Shameems> db.createCollection("Customers")
{ ok: 1 }
Shameems> db.createCollection("Orders")
{ ok: 1 }
Shameems> db.createCollection("Employees")
{ ok: 1 }
Shameems> db.createCollection("Promotion")
{ ok: 1 }
Shameems> db.createCollection("Products")
{ ok: 1 }
Shameems> db.createCollection("User")
{ ok: 1 }
Shameems>
```

Figure 4.11: Create database collections

Create collections with their respective names.

Checking Created Collections

show collections



```
> show collections
< Customers
  Employees
  Orders
  Products
  Promotion
  User
Shameems > |
```

Figure 4.12: Show database collections

Lists all collections within the current database.

Inserting Documents into Collections



```
> db.Customers.insertOne({
  customer_id: "CUST001",
  name: "Upali Rathnayake",
  email: "upali02@gmail.com",
  phone: 772349729,
  gender: "Male",
  loyalty_points: 10,
})
< {
  acknowledged: true,
  insertedId: ObjectId('68aaa2f44a8d12ddcdcc3ccd')
}
```

Figure 4.13: Inserting a Single Document into a Collection 'Customers'

Once collections are created, data insertion is achievable. Creating a single document is achieved through such commands.

Inserting Many Documents into Collections

```
> db.Customers.insertMany([
  {
    "customer_id": "CUST002",
    "name": "Nazeera Fathima",
    "email": "naz@gmail.com",
    "phone": 771991267,
    "gender": "Female",
    "loyalty points": 25
  },
  {
    "customer_id": "CUST003",
    "name": "Imran Ahamed",
    "email": "imran@gmail.com",
    "phone": 772223344,
    "gender": "Male",
    "loyalty points": 5
  },
])
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('68aaa4ef4a8d12ddcdcc3cce'),
    '1': ObjectId('68aaa4ef4a8d12ddcdcc3ccf')
  }
}
```

Figure 4.14: Inserting a Many Documents into a Collection 'Customers'

Multiple document insertion is achieved through such commands.

Querying Data in Collections

```
db.Customers.find()
```

```
> db.Customers.find()
< {
  _id: ObjectId('68aaa2f44a8d12ddcdcc3ccd'),
  customer_id: 'CUST001',
  name: 'Upali Rathnayake',
  email: 'upali02@gmail.com',
  phone: 772349729,
  gender: 'Male',
  loyalty_points: 10
}
{
  _id: ObjectId('68aaa4ef4a8d12ddcdcc3cce'),
  customer_id: 'CUST002',
  name: 'Nazeera Fathima',
  email: 'naz@gmail.com',
  phone: 771991267,
  gender: 'Female',
  'loyalty points': 25
}
{
  _id: ObjectId('68aaa4ef4a8d12ddcdcc3ccf'),
  customer_id: 'CUST003',
  name: 'Imran Ahamed',
  email: 'imran@gmail.com',
  phone: 772223344,
  gender: 'Male',
  'loyalty points': 5
}
```

Figure 4.15: Find Documents in Collection 'Customers'

Returns all customer records in the 'Customers' collection

Updating Documents in Collections

```
> db.Products.updateOne(
  {product_id: "PRD001"},
  {$set: {price: 1750}}
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Figure 4.16: Update price in Collection 'Products'

Change the price of the product using Product ID to 1750

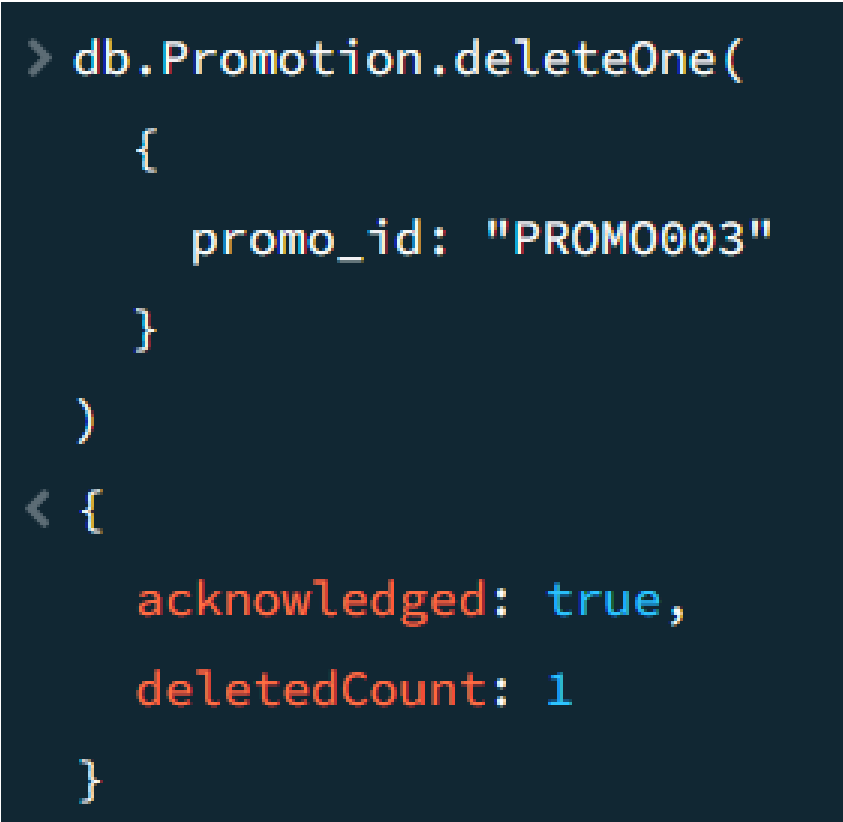
```
> db.Products.updateOne(
  { product_id: "PRD001" },
  { $inc: { stock: -2 } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Figure 4.17: Update the stock in Collection 'Products'

Change the stock of the product using Product ID to -2

Deleting Documents from Collections

```
db.Promotion.deleteOne({ promo_id: PROM0003 })
```

A screenshot of a MongoDB shell session. The first line shows a command to delete a document from the 'Promotion' collection where 'promo_id' is 'PROM0003'. The second line shows the output of the command, which is a JSON object indicating the deletion was successful, with 'acknowledged' set to true and 'deletedCount' set to 1.

```
> db.Promotion.deleteOne(
  {
    promo_id: "PROM0003"
  }
)
< {
  acknowledged: true,
  deletedCount: 1
}
```

Figure 4.18: Delete the Promotion record of 'PROMO003' in 'Promotions' collection

4.5 MongoDB Compass

MongoDB Compass offers a graphical user interface (GUI) to developers and administrators to use the database without the need to write shell commands. It simplifies the development, visualization, and debugging process by accessing collections, documents, and queries that are easily accessible by clicking.

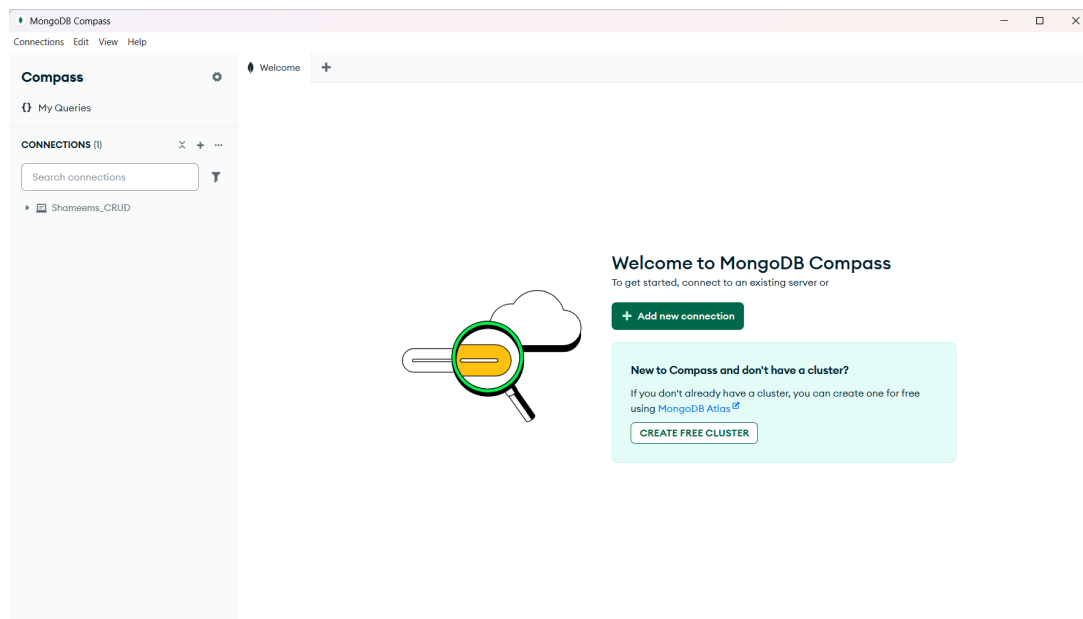


Figure 4.19: User Interface of MongoDB Compass

The features of Compass include database creation, schema exploration, query building, index management, and graphical representation of data distribution. It is particularly handy for users who are visual interaction-oriented instead of command-line-oriented.

4.5.1 Database Development in MongoDB

The following steps explain how the Shameems Cosmetics database was developed using MongoDB Compass:

1. **Creating a New Connection:** Using the Compass interface, connect to the MongoDB server by entering the connection string (e.g., `mongodb://localhost:27017`). This establishes access to manage databases.

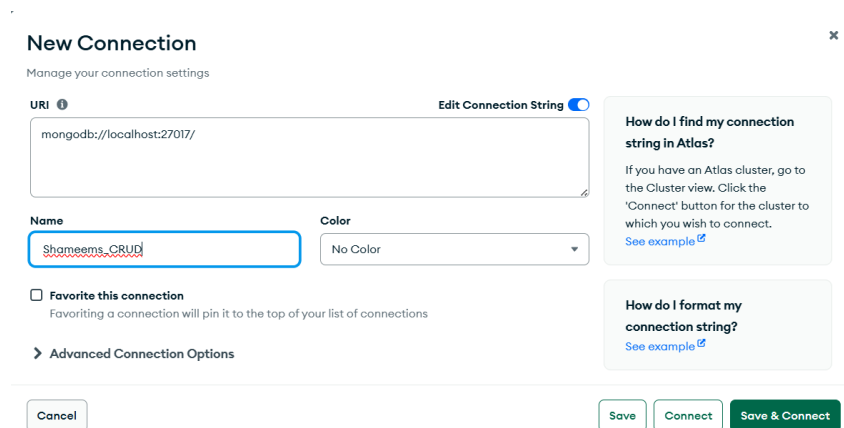
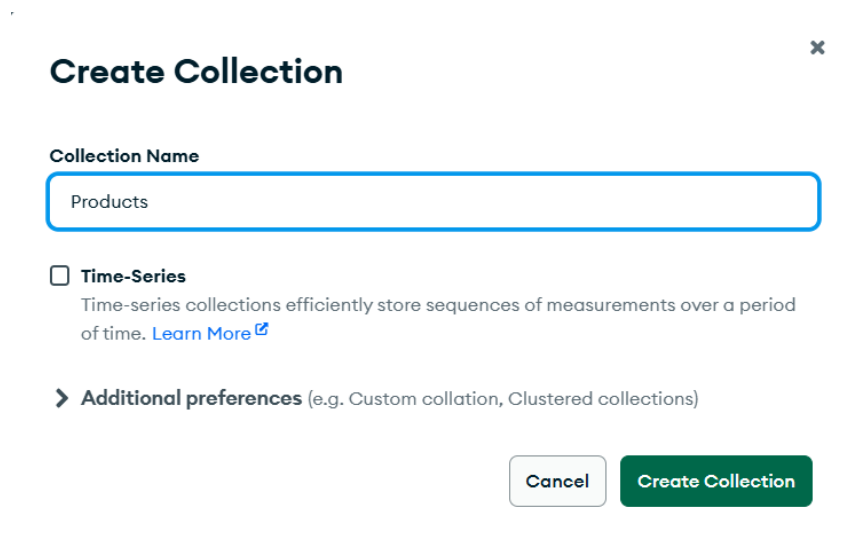


Figure 4.20: Creating a new connection in MongoDB Compass

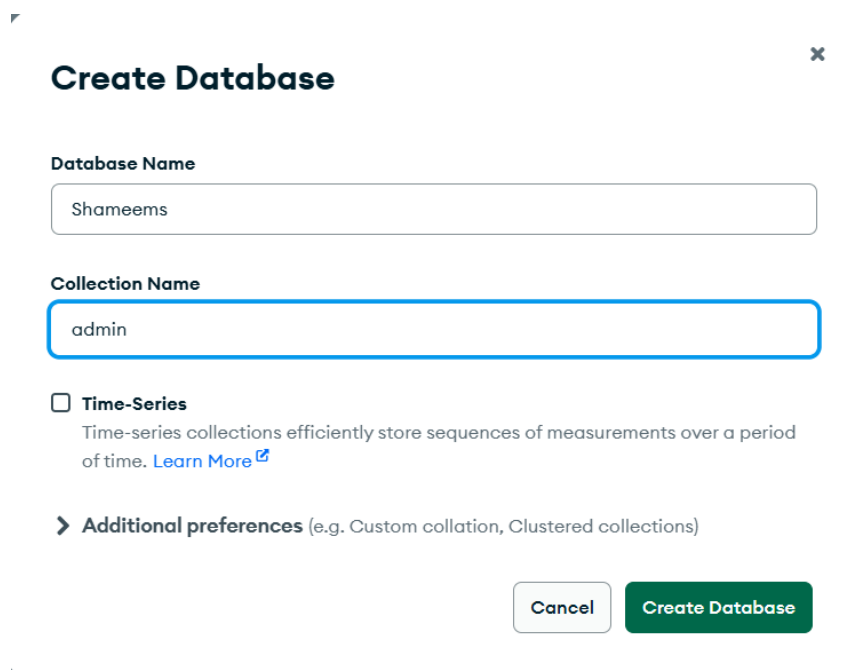
2. **Creating a New Collection:** Within the `shameems` database, additional collections such as `orders` or `promotions` can be created to organize data by business functionality.



The screenshot shows the 'Create Collection' dialog box in MongoDB Compass. At the top, the title 'Create Collection' is displayed in bold, with a close button (X) to its right. Below the title, there is a 'Collection Name' label followed by a text input field containing the word 'Products'. Underneath this, there is a checkbox labeled 'Time-Series' which is currently unchecked. To the right of the checkbox, there is a descriptive text: 'Time-series collections efficiently store sequences of measurements over a period of time. [Learn More](#)'. Below this, there is a section titled 'Additional preferences' with a right-pointing arrow and a subtitle '(e.g. Custom collation, Clustered collections)'. At the bottom right of the dialog, there are two buttons: a light gray 'Cancel' button and a dark green 'Create Collection' button.

Figure 4.21: Creating a new collection in MongoDB Compass

3. **Creating a Database:** In Compass, choose the option "Create Database" and name it `Shameems` and then create the first collections such as `customers`, `employees` and `admin`.



The screenshot shows the 'Create Database' dialog box in MongoDB Compass. At the top, the title 'Create Database' is displayed in bold, with a close button (X) to its right. Below the title, there is a 'Database Name' label followed by a text input field containing the word 'Shameems'. Underneath this, there is a 'Collection Name' label followed by a text input field containing the word 'admin'. Below this, there is a checkbox labeled 'Time-Series' which is currently unchecked. To the right of the checkbox, there is a descriptive text: 'Time-series collections efficiently store sequences of measurements over a period of time. [Learn More](#)'. Below this, there is a section titled 'Additional preferences' with a right-pointing arrow and a subtitle '(e.g. Custom collation, Clustered collections)'. At the bottom right of the dialog, there are two buttons: a light gray 'Cancel' button and a dark green 'Create Database' button.

Figure 4.22: Creating a Database in MongoDB Compass

4. **Adding Documents:** Using the "Insert Document" option, JSON-like documents can be manually added to collections.

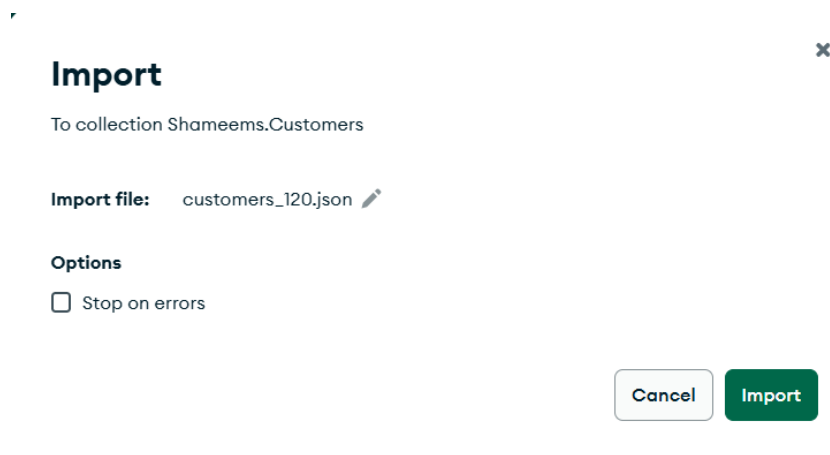


Figure 4.23: Importing Documents in the format of JSON or CSV files

5. **View Inserted Documents:** After importing or adding documents the record will be visible in the User Interface.

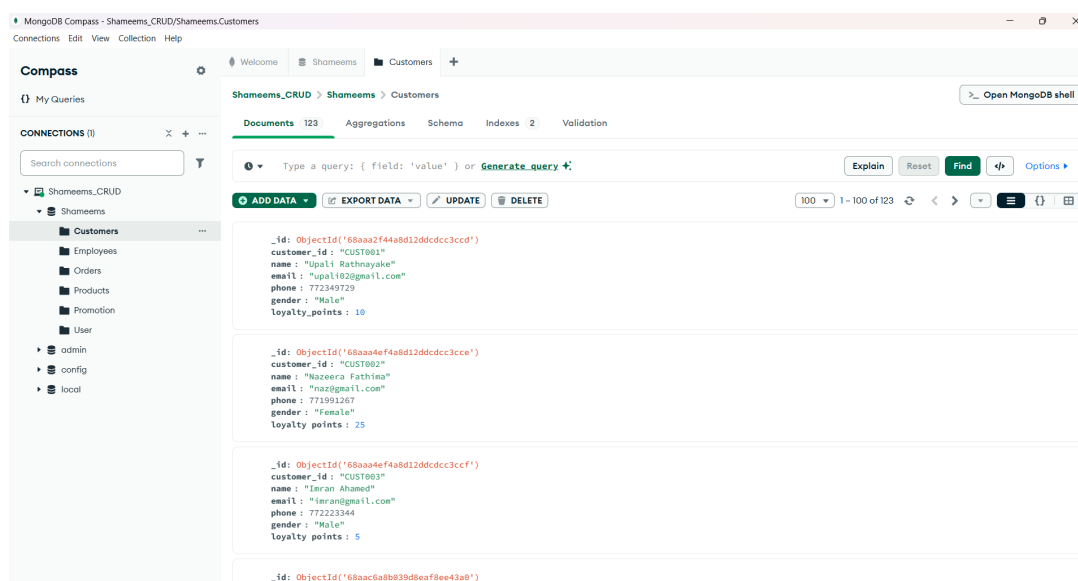


Figure 4.24: View the inserted records

6. **Editing Documents:** Fields such as price, stock, or loyalty points can be updated directly within Compass.

```

1  _id: ObjectId('68aaa2f44a8d12ddccc3ccd')
2  customer_id: "CUST001"
3  name: "Upali Rathnayake"
4  email: "upali02@gmail.com"
5  phone: 772349329
6  gender: "Male"
7  loyalty_points: 10

```

Document modified.

CANCEL UPDATE

Figure 4.25: Updating phone number Field

7. **Finding Documents:** Queries can be built using filter options, e.g. `{ "Categories": "Men's Perfumes" }`, to fetch relevant data.

Shumair > Shameems > Products

Documents 150 Aggregations Schema Indexes 2 Validation

{ category: "Men's Perfumes" }

Generate query Explain Reset Find Options

ADD DATA EXPORT DATA UPDATE DELETE

100 1 - 12 of 12

```

_id: ObjectId('68b457a158bf7f897a9a2631')
product_id: "PRD003"
name: "Eros 50ml"
category: "Men's Perfumes"
brand: "ChandanaLepa"
price_lkr: 21500
stock: 17
expiry_date: 2028-07-31T00:00:00.000+00:00

```

```

_id: ObjectId('68b457a158bf7f897a9a263c')
product_id: "PRD014"
name: "One 50ml"
category: "Men's Perfumes"
brand: "Neutrogena"
price_lkr: 25000
stock: 28
expiry_date: 2027-04-02T00:00:00.000+00:00

```

```

_id: ObjectId('68b457a158bf7f897a9a264d')
product_id: "PRD031"
name: "One 50ml"
category: "Men's Perfumes"
brand: "Maybelline"
price_lkr: 16000
stock: 24
expiry_date: 2026-02-23T00:00:00.000+00:00

```

Figure 4.26: Find documents with filter options

8. **Deleting Documents:** Records can be removed with the delete function, either individually or in bulk.

```

_id: ObjectId('68aaa2f44a8d12ddccc3ccd')
customer_id: "CUST001"
name: "Upali Rathnayake"
email: "upali02@gmail.com"
phone: 772349729
gender: "Male"
loyalty_points: 10

```

Document flagged for deletion.

CANCEL DELETE

Figure 4.27: Delete a specific document

9. **Drop Collection:** Entire collections can be deleted using the "Drop Collection" option in the GUI.

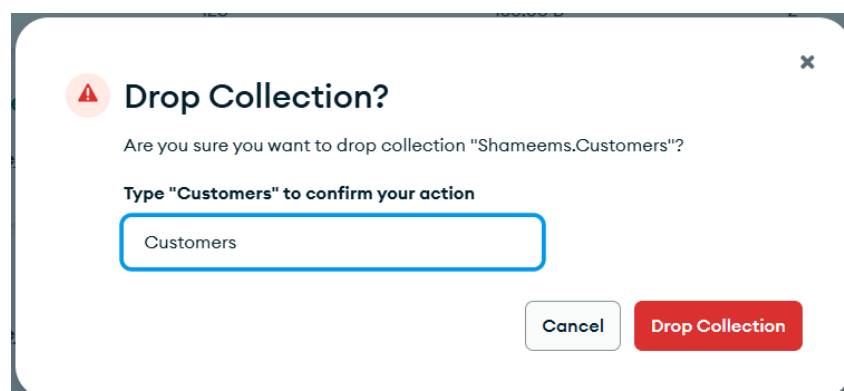


Figure 4.28: Drop a specific collection

The database design and development process, executed through MongoDB Shell and Compass, ensured structured creation, manipulation, and management of data. By combining command-line precision with GUI-based simplicity, the Shameems Cosmetics system achieves flexibility, scalability, and reliability, forming the foundation for efficient analytics and informed business decision-making.

Chapter 5

MERN Stack Development

5.1 Overview

The MERN stack is a popular web development framework comprising MongoDB, Express.js, React.js, and Node.js. It provides a full JavaScript-based environment, enabling rapid application development with scalability, flexibility, and seamless integration with front-end and back-end layers. This architecture is particularly suited for dynamic, data-driven applications like Shameems Cosmetics.

5.1.1 How the MERN Stack Works

- **MongoDB:** Stores documents that contains the data of an application in the form of a JSON.
- **Express.js:** The server is the backbone repository, which will construct APIs that link the database to the client side.
- **React.js:** Offers a user interface of dashboards and forms, real-time data rendering.
- **Node.js:** Executes server-side JavaScript to handle business logic and process client requests.

5.1.2 Advantages of MERN Stack

- **Full JavaScript Development:** Removes the necessity of switching languages making development more straightforward.
- **Scalability:** MongoDB and Node.js provide a horizontal scaling system to bigger datasets and simultaneous requests.

- **High Performance:** React.js is a fast renderer, whereas the Node.js handles asynchronous operation.
- **Flexibility:** MERN facilitates the easy code design (modular design), and thus expansion of a system is easy.
- **Open-Source Ecosystem:** Project development is fast-tracked by large developer community and possessing readily available libraries.
- **Cross-Platform Development:** The apps written in MERN can be ported to various platforms such as cloud services or containerized environments, among others.
- **Rapid Prototyping:** The stack's unified environment allows developers to quickly build and test prototypes before moving into production.

5.2 Shameems Cosmetics Dashboard System

The Shameems Cosmetics Management System dashboard serves as the central interface to monitor business operations. It displays key metrics such as total customers, products, orders, and active promotions. The dashboard also provides navigation to dedicated pages for managing collections, enabling quick access and real-time data updates that support informed business decision-making.

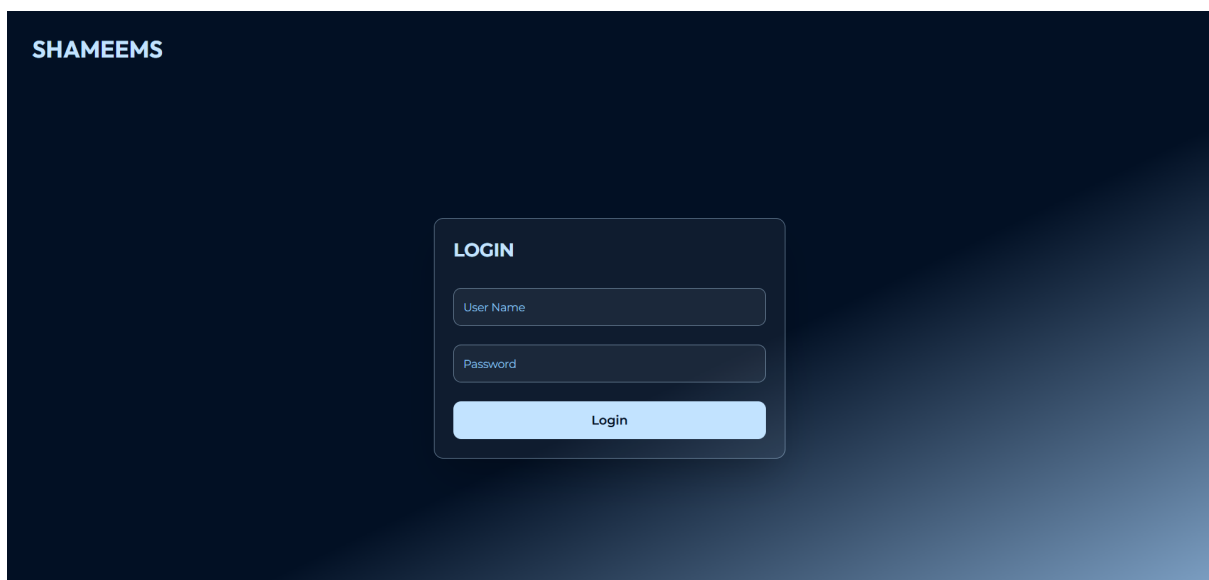


Figure 5.1: Shameems Cosmetics Login Page

5.2.1 Interfaces

Dashboard Interface

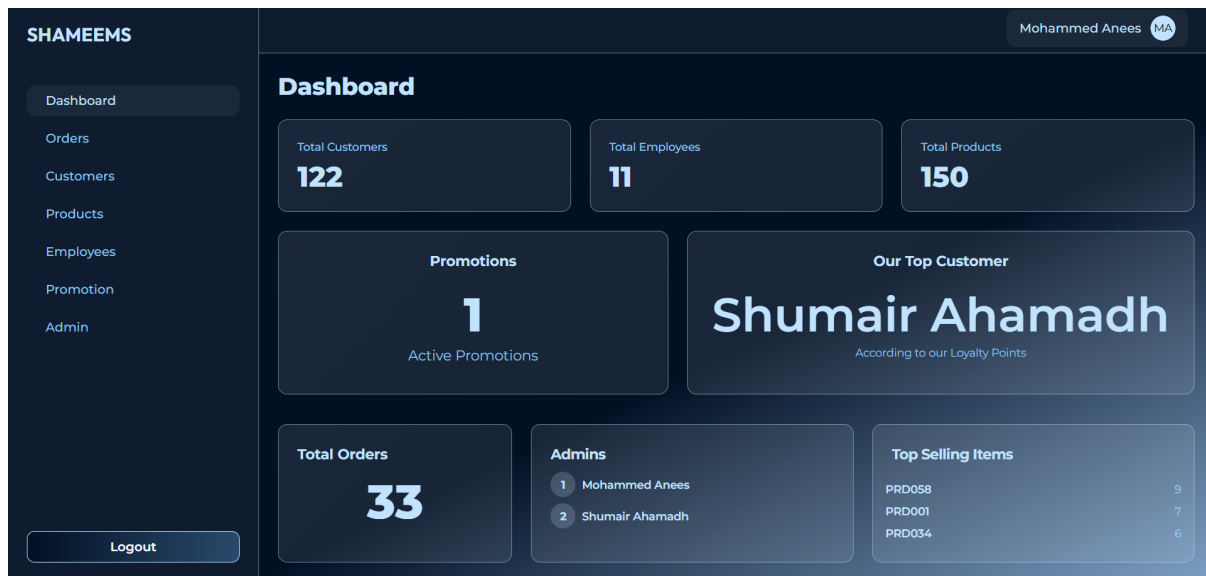


Figure 5.2: Shameems Cosmetics Dashboard

These cards are displayed as an overview of the business.

- Total Customers
- Total Employees
- Total Products
- Total Orders
- Active Promotions
- Top Customer
- Top-Selling Products

Functions:

- **View Metrics:** Provides an overview of business operations at a glance.
- **Navigation:** Acts as a gateway to other interfaces for CRUD operations.

Customers Interface

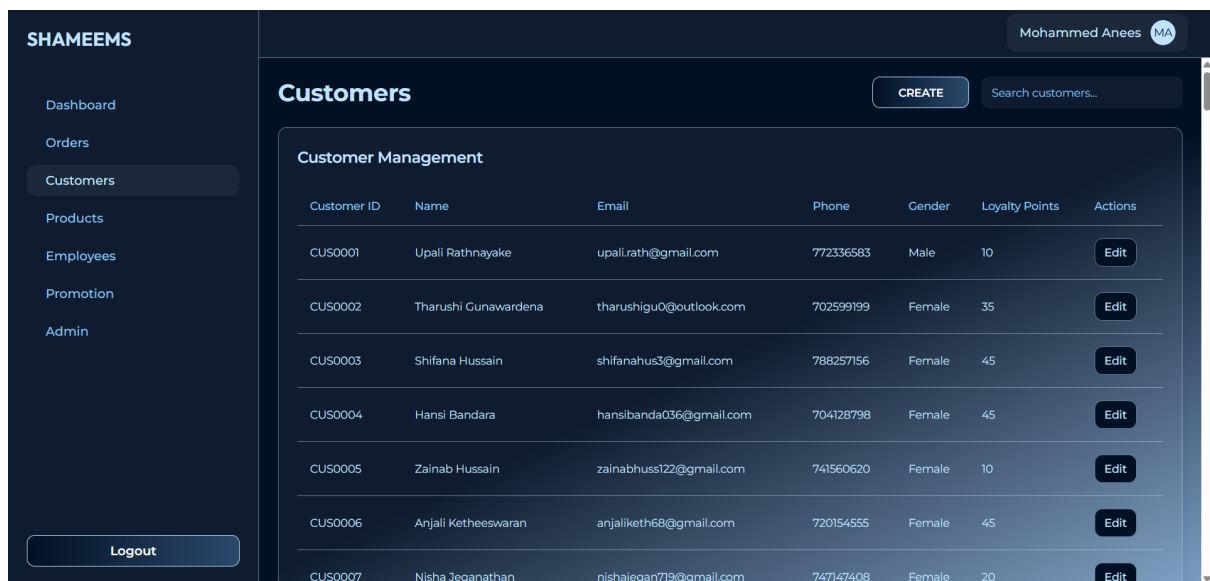


Figure 5.3: Shameems Cosmetics Customers Page

Functions:

- **Create:** Add new customers with personal details and contact information.
- **Edit:** Update loyalty points or customer details when necessary.
- **Delete:** Remove inactive or duplicate customer records.
- **Search:** Locate customer records using name, phone, or email filters.

Orders Interface

Orders ID	Customer ID	Product ID	Quantity	Price (LKR)	Payment	Promo ID	Date	Created	Total (LKR)	Actions
ORD001	CUS0016	PRD027	2	1800	Card	NO	19/08/2025	EMP002	28050	Edit
		PRD058	3	1050						
		PRD143	3	7100						
ORD002	CUS0097	PRD001	3	4100	Cash	NO	13/08/2025	EMP003	12300	Edit
ORD003	CUS0015	PRD003	3	27000	Cash	PROMO004	10/07/2025	EMP002	76050	Edit
		PRD126	2	3600						
ORD004	CUS0051	PRD097	3	1750	Card	NO	03/08/2025	EMP002	7050	Edit
		PRD132	1	1800						
ORD005	CUS0012	PRD074	1	24500	Card	PROMO004	22/08/2025	EMP003	27575	Edit
		PRD116	3	2250						
ORD006	CUS0004	PRD119	1	1450	Card	NO	06/08/2025	EMP003	3650	Edit
		PRD109	2	1100						

Figure 5.4: Shameems Cosmetics Order Page

Functions:

- **Create:** Generate new orders by selecting products, customers, and promotions.
- **Edit:** Update quantities or payment method before finalizing orders.
- **Delete:** Cancel orders when mistakes occur or stock is unavailable.
- **Search:** Retrieve orders by ID, customer name, or date.

Products Interface

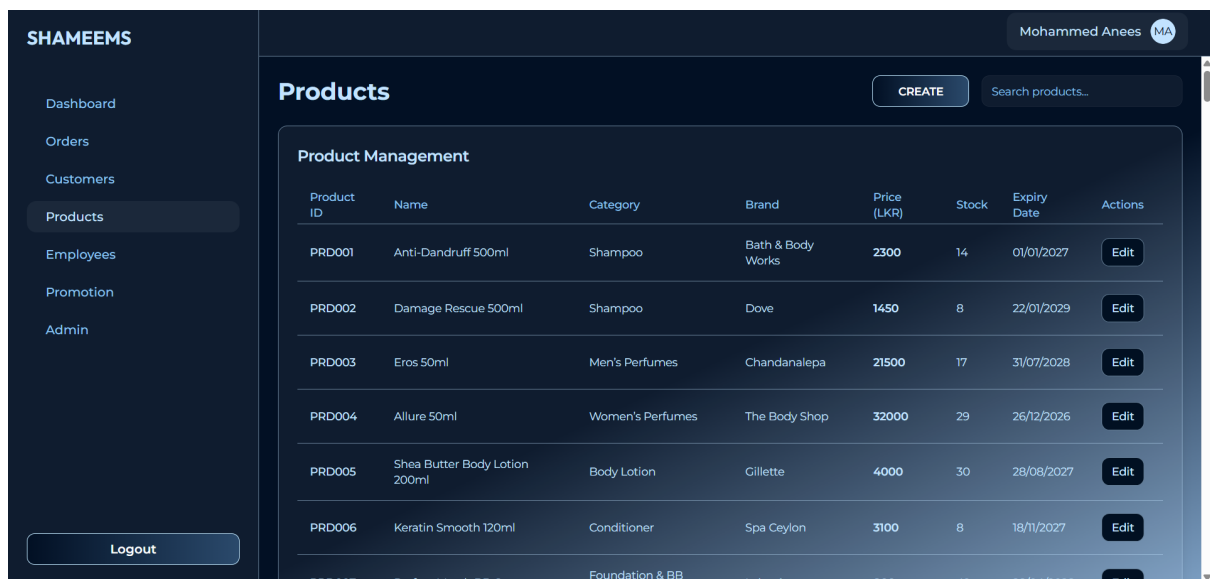


Figure 5.5: Shameems Cosmetics Products Page

Functions:

- **Create:** Add new products with pricing and expiry details.
- **Edit:** Update stock levels or modify product information.
- **Delete:** Remove expired or discontinued items.
- **Search:** Find products by category, brand, or expiry date.

Employees Interface

SHAMEEMS

Dashboard

Orders

Customers

Products

Employees

Promotion

Admin

Logout

Mohammed AneesMA

Employees

CREATE

Search employees...

Employee Management

Employee ID	Full Name	Email	Phone	Role	Salary (LKR)	Status	Actions
EMP001	Supun Bandara	supunbandara@outlook.com	759081935	Cashier	34,000	active	Edit
EMP002	Rizwan Habeeb	rizwanhabeeb@gmail.com	719375836	Sales	33,000	active	Edit
EMP003	Mohammed Hussain	mohd.03hameerahussain@gmail.com	787457446	Cashier	32,000	active	Edit
EMP004	Shenali Rodrigo	shenalirodrigo@hotmail.com	729361589	Sales	21,000	left	Edit
EMP005	Amal Perera	amalperera@gmail.com	759982352	Sales	28,000	left	Edit
EMP006	Nuwan Silva	nuwansilva@gmail.com	772492024	Accountant	22,000	left	Edit
EMP007	Dinusha	dinushaweerasinghe@abon.com	737058365	Cashier	24,000	left	Edit

Figure 5.6: Shameems Cosmetics Employees Page

Functions:

- **Create:** Add new employees with role and salary details.
- **Edit:** Modify records to reflect promotions, salary changes, or role shifts.
- **Delete:** Remove records of employees who have resigned or been terminated.
- **Search:** Search employees by role, name, or employment status.

Promotions Interface

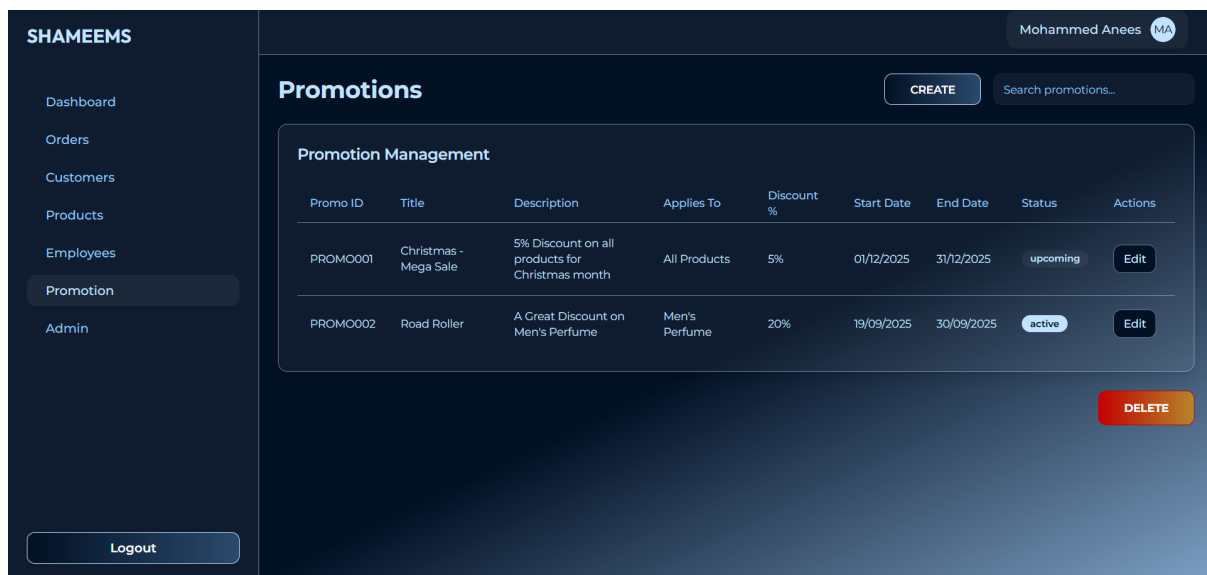


Figure 5.7: Shameems Cosmetics Promotions Page

Functions:

- **Create:** Add new promotion with title, description, value along with the start and end dates.
- **Edit:** Modify records to reflect promotions status.
- **Delete:** Remove records of promotions which have been expired.
- **Search:** Search promotions by title and value.

Admins Interface

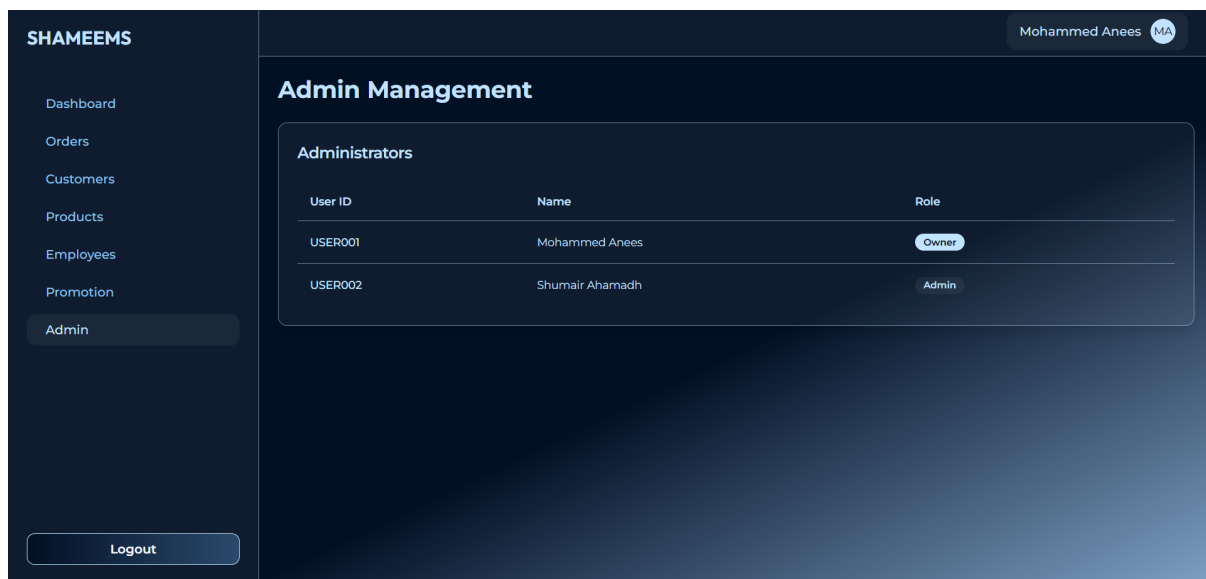


Figure 5.8: Shameems Cosmetics Admin Page

Functions:

- **View Only:** Admin records are restricted to viewing only for security and confidentiality. No create, edit, or delete operations are permitted.

The MERN stack enabled the development of a scalable, efficient, and user-friendly management system. With CRUD interfaces and real-time analytics, Shameems Cosmetics can manage customers, employees, products, and orders effectively while ensuring data integrity, security, and business growth through digital transformation.

Chapter 6

Data Analysis and Insights

6.1 Relevance to the Shameems Cosmetics

Data analysis plays a vital role in enabling businesses to make informed decisions and maintain customer loyalty in a competitive retail environment. For Shameems Cosmetics, analytics are used to convert raw transactional and operational data into actionable insights that can be used to support strategic objectives. With the help of the MongoDB aggregation pipelines, managers can know the sales tendencies, monitor the most selling goods, and estimate the efficiency of promotions. To track high-value customers, loyalty points can be tracked and data on employee performance will hold them responsible. In addition, expiry tracking and stock-level analysis minimize losses and optimize inventory planning.

Without structured analysis, business risks inefficiencies such as expired stock, missed sales opportunities, and poor customer retention. Therefore, insights derived from analytics not only solve immediate operational issues, but also provide a foundation for future expansion, digital marketing integration, and predictive business intelligence.

6.2 Solving the Business Problem

To overcome the challenges mentioned earlier, Shameems Cosmetics uses MongoDB-based analytics to turn data into insights. The system will also eliminate poor record keeping, expiration losses, and customer tracking, making the decision-making process and operational efficiency smarter.

6.2.1 Customer Retention

Analytics identify repeat customers through loyalty points and purchase history. The business can also establish better relationships with them through targeted promotions, which will result in higher customer loyalty.

6.2.2 Stock Management

Product stock and expiry date are monitored by real time queries. This helps to avoid stock- outs or overstocking and minimize financial losses in stocked goods.

6.2.3 Sales Trend

The sales analysis will enable the management to know when to engage in promotion and how to stock to ensure that the demand is met.

6.2.4 Employees Performance

Employee records will contain order records that will enable the business to monitor sales that are processed by every employee. This assists in determining the training requirements and in rewarding the performers.

6.2.5 Promotional Effectiveness

Promotions can be evaluated by linking orders with discount campaigns. This identifies the most effective promotions to increase sales and customer interaction.

6.2.6 Financial Insights

The mean order value analytics, active promotions, and cost of losses on expired stock assist the owner to assess profitability and make financial decisions based on the information.

6.3 Key Questions and Queries

To guide effective analytics, the following business operations were developed. These will be answered using MongoDB queries executed in `mongosh`

6.3.1 Question 1: What are the top-selling products?

Method: `aggregate()` with `$unwind`, `$group`, and `$sort`.

Objective: Identify the best selling products to maintain stocks and manage t promotions.

Explanation: Aggregation is applied to tally the number of sales of products per order to assist the management in determining that demand and focusing on inventory on products that are doing well.

```
> db.Order.aggregate([
  {$unwind: "$items"},
  {$group: {_id: "$items.product_id", totalSold: {$sum: "$items.quantity" } } },
  {$sort: {totalSold: -1} },
  {$limit: 5}
])
```

Figure 6.1: Question 1. Top-selling Products (Query)

```
< {  
  _id: 'PRD058',  
  totalSold: 9  
}  
{  
  _id: 'PRD001',  
  totalSold: 7  
}  
{  
  _id: 'PRD034',  
  totalSold: 6  
}  
{  
  _id: 'PRD116',  
  totalSold: 6  
}  
{  
  _id: 'PRD027',  
  totalSold: 5  
}
```

Figure 6.2: Answer 1. Top-selling Products (Query)

6.3.2 Question 2. Who are the most loyal customers based on the loyalty points?

Method: find() with \$sort.

Objective: Reward and retain top customers based on loyalty points.

Explanation: The grouping of the customers in terms of loyalty points identifies the

most active customers, which can be used in order to retain them with discounts or personalized offers.

```
> db.Customers.find().sort({ loyalty_points: -1 }).limit(3)
< {
  _id: ObjectId('68ceb21af26549a03cf35ece'),
  customer_id: 'CUST121',
  name: 'Shumair Ahamadh',
  email: 'shumair0304@gmail.com',
  phone_lk: 766603266,
  gender: 'Male',
  loyalty_points: 50
}
{
  _id: ObjectId('68b4558358bf7f897a0a25b9'),
  customer_id: 'CUS0006',
  name: 'Anjali Ketheeswaran',
  email: 'anjaliketh68@gmail.com',
  phone_lk: 720154555,
  gender: 'Female',
  loyalty_points: 45
}
{
  _id: ObjectId('68b4558358bf7f897a0a25b7'),
  customer_id: 'CUS0004',
  name: 'Hansi Bandara',
  email: 'hansibanda036@gmail.com',
  phone_lk: 704128798,
  gender: 'Female',
  loyalty_points: 45
}
```

Figure 6.3: Question 2. Loyal Customers based on loyalty points

6.3.3 Question 3. What is the total number of customers?

Method: `countDocuments()`.

Objective: Measure the active customer base to assess business growth.

Explanation: Counting active customers provide insight into the store's engagement level, displaying whether the customer base is increasing or declining.

```
> db.Employees.countDocuments({ status: "active" })
< 4
```

Figure 6.4: Question 3. Total number of customers.

6.3.4 Question 4. How many employees are currently active, and what roles do they perform?

Method: `aggregate()` with `$group`.

Objective: Evaluate workforce distribution accross roles.

Explanation: Grouping active employees by role highlights and staffing structure, ensuring balanced workforce allocation between cashier and sales.

```
> db.Employees.aggregate([
  {$match: {status: "active"} },
  {$group: { _id: "$role", total: {$sum: 1 } } }
])
< {
  _id: 'Cashier',
  total: 2
}
{
  _id: 'Sales',
  total: 2
}
```

Figure 6.5: Question 4. Active employees and roles.

6.3.5 Question 5. Which product categories have the highest average price?

Method: `aggregate()` with `$group` and `$avg`.

Objective: Identify product categories which contribute to higher margins.

Explanation: By grouping the products according to the category and calculating the average unit price, the management can then know which categories are of higher value. This facilitates targeted marketing, pricing policies and aids in balancing of stock between high-end products and low-end products.

```
> db.Products.aggregate([
  {
    $group: {
      _id: "$category",
      avgPrice: { $avg: "$price_lkr" },
      productCount: { $sum: 1 }
    }
  },
  { $sort: { avgPrice: -1 } },
  {$limit: 3}
])
< {
  _id: 'Men's Perfumes',
  avgPrice: 29270.833333333332,
  productCount: 12
}
{
  _id: 'Women's Perfumes',
  avgPrice: 20850,
  productCount: 5
}
{
  _id: 'Compact & Powder',
  avgPrice: 7785,
  productCount: 10
}
```

Figure 6.6: Question 5. Products with high average price.

6.4 Hypothesis

6.4.1 Hypothesis 1: Promotions increase order frequency within a period

Method: aggregate() with \$match (date range), \$project (hasPromo), \$group, \$project (uplift).

Objective: Prove that promotions drive more orders to justify promotional spend.

Explanation: We isolate a period, flag each order as promo/non-promo, and count both groups. Comparison of counts demonstrates whether promos are related to increased frequency of order. A simple uplift % can also be calculated to measure the effect. In case promo orders significantly outnumber non-promo orders (other things held constant over the same time period), the hypothesis is upheld.

```
Shameems> db.Order.aggregate([
  {
    $match: {
      order_date: { $gte: ISODate("2025-01-01"), $lte: ISODate("2025-10-31") }
    }
  },
  {
    $project: {
      hasPromo: { $cond: [{ $ifNull: ["$promo_id", false] }, true, false] }
    }
  },
  {
    $group: {
      _id: "$hasPromo",
      orders: { $sum: 1 }
    }
  },
  {
    $project: {
      _id: 0,
      group: { $cond: ["$_id", "Promo", "NoPromo"] },
      orders: 1
    }
  }
])
```

Figure 6.7: Hypothesis 1: Question

```
< {  
  orders: 7,  
  group: 'Promo'  
}  
  
{  
  orders: 26,  
  group: 'NoPromo'  
}
```

Figure 6.8: Hypothesis 1: Answer

Interpretation

- Only 21% of the orders use the promotion.
- Therefore, the hypothesis is not supported, since sales did not increase in the promotion period.

6.4.2 Hypothesis 2: Certain product categories generate the highest revenue?

Method: `aggregate()` with `$unwind (items)`, `$lookup (join products)`, `$group by category`, then `sum price × quantity`.

Objective: Identify which generates the most sales and revenue.

Explanation: We join order items and the product categories and multiply the quantity $\times$ price to get revenue per category. The ranking of them will show what types of categories contribute the most revenue. When some categories are predominant, the management can make them the priority of promotions and keep them well-stocked.

```
Shameems> db.Order.aggregate([
  { $unwind: "$items" },
  { $addFields: { qty: { $toInt: "$items.quantity" }, priceNum: { $toDouble: "$items.price" } } },
  { $lookup: { from: "Products", localField: "items.product_id", foreignField: "product_id", as: "prod" } },
  { $unwind: "$prod" },
  { $group: { _id: "$prod.category", totalRevenue: { $sum: { $multiply: ["$qty", "$priceNum"] } }, itemsSold: { $sum: "$qty" } } },
  { $project: { category: "$_id", totalRevenue: { $round: ["$totalRevenue", 2] }, itemsSold: 1, _id: 0 } },
  { $sort: { totalRevenue: -1 } }
])
```

Figure 6.9: Hypothesis 2: Question

```
< {
  itemsSold: 9,
  category: 'Moisturizer & Cream',
  totalRevenue: 160300
}
{
  itemsSold: 9,
  category: 'Men's Perfumes',
  totalRevenue: 152500
}
{
  itemsSold: 7,
  category: 'Face Wash & Cleanser',
  totalRevenue: 72450
}
{
  itemsSold: 23,
  category: 'Shampoo',
  totalRevenue: 55800
}
{
  itemsSold: 6,
  category: 'Conditioner',
  totalRevenue: 42500
}
```

Figure 6.10: Hypothesis 2: Answer

Interpretation

- The top revenue categories are Moisturizer & Cream and Men's Perfumes, which are higher than the rest.
- Even though shampoo has sold the most units, revenue is much lower due to lower price per unit.

- This confirms the hypothesis, revenue is not evenly spread according to the categories. As some products generate more revenue even with fewer items sold.

6.4.3 Hypothesis 3. Top brands hold atleast 50% of the total stock units?

Method: aggregate() with \$group, \$sort, \$limit, plus a second-stage \$group to compare Top-3 stock vs. Total stock and compute a percentage.

Objective: Check if the inventory is concentrated in a few brands (Dove-like), guiding the purchase, shelf space, and risk management.

Explanation: We sum stock units per brand, identify the top three by stock, and compare their combined units to the grand total. Use of Top-3 share $\geq 50\%$ inventory, the business has concentration, and this means that it has dependency risk and recourse to re balance assortment, negotiate with key suppliers, or diversify into underrepresented brands.

```
Shameems> db.Products.aggregate([
  { $group: { _id: "$brand", units: { $sum: { $toInt: "$stock" } } } },
  { $sort: { units: -1 } },

  { $group: { _id: null, allBrands: { $push: { brand: "$_id", units: "$units" } } }, top3: { $push: { brand: "$_id", units: "$units" } } } },
  { $project: { top3: { $slice: ["$top3", 3] }, allBrands: 1 } } },
  { $project: { top3Units: { $sum: "$top3.units" }, totalUnits: { $sum: "$allBrands.units" } } },
  { $project: { top3Units: 1, totalUnits: 1, top3SharePct: {
    $cond: [ { $gt: ["$totalUnits", 0] }, { $multiply: [ { $divide: ["$top3Units", "$totalUnits" ] }, 100 ] }, null ]
  }
  }
  ]
})
```

Figure 6.11: Hypothesis 3: Question

```
< {
  _id: null,
  top3Units: 664,
  totalUnits: 3738,
  top3SharePct: 17.763509898341358
}
```

Figure 6.12: Hypothesis 3: Answer

Interpretation

- The actual share 17.76% is well below 50%.

- Therefore, the hypothesis is rejected.

Chapter 7

Conclusion

The project provided insight into the ways in which Shameems Cosmetics could redesign its old-fashioned record-keeping systems into new data-driven management systems. Creating a MongoDB database that contains collections of customers, employees, products, orders, promotions, and administrators, the system provides effective storage and retrieval of business information. The MERN stack was used to create a working dashboard and interfaces that are used to handle functions like order tracking, stocks monitoring, employee management, and promotions.

The integration of data analysis added further value by answering key business questions through aggregation pipelines and queries. An understanding of the best-selling products, revenue by category, performance of employees, and effectiveness of promotion was created to support decision making. The assumptions were also tested or disproved by hypothesis testing, which helped the business better understand its business.

Overall, this solution offers Shameems Cosmetics a scalable, flexible and reliable platform to conduct its daily business and long term expansion. It demonstrates the significance of integrating the concept of modern database design and analytics in order to enhance efficiency, minimize losses, and propel profitability.

Bibliography

- Chodorow, Kristina (2013). *MongoDB: The Definitive Guide*. 2nd. Sebastopol, CA: O'Reilly Media.
- Coronel, Carlos and Steven Morris (2019). *Database Systems: Design, Implementation, & Management*. 13th. Boston, MA: Cengage Learning.
- Gonzalez, James, Chris White, and John Davis (2020). *Data Pipelines Pocket Reference*. Sebastopol, CA: O'Reilly Media.
- Laudon, Kenneth C. and Carol G. Traver (2023). *E-Commerce 2023: Business, Technology and Society*. 18th. London: Pearson Education.
- MongoDB Inc. (2024). *What is the MERN Stack?* <https://www.mongodb.com/mern-stack>. Accessed 21 May 2025.
- Provost, Foster and Tom Fawcett (2013). *Data Science for Business: What You Need to Know About Data Mining and Data-Analytic Thinking*. Sebastopol, CA: O'Reilly Media.
- Sadalage, Pramod J. and Martin Fowler (2012). *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Boston, MA: Addison-Wesley.
- Stonebraker, Michael (2010). "SQL Databases v. NoSQL Databases". In: *Communications of the ACM* 53.4, pp. 10–11.